sql-general

General SQL Assistant Prompt Chain

Use Case

Use this chain as the everyday Oracle SQL entry point for Copilot. It helps the analyst classify the request, collect missing context, route specialized cases, and produce a predictable final answer.

Use this chain when the task is not clearly one of the specialized workflows yet.

Supported Task Types

- Write new SQL.
- Explain existing SQL.
- Refactor SQL.
- Debug a SQL error.
- Improve query readability.
- Improve query efficiency.
- Map business logic to fields and tables.

Routing Guidance

- Use `prompt-chains/sql-efficiency/` when the main goal is faster SQL from a business requirement.
- Use `prompt-chains/sql-logic-mapping/` when the main goal is clarifying logic, table, and field mappings before implementation.
- Use `procedures/query_tuner/SKILL.md` or `prompt-chains/query-tuning/` when `EXPLAIN PLAN` based tuning is needed.

Prompt Sequence

1. `01-task-classification.md`: classify the SQL task and route specialized cases.
2. `02-context-intake.md`: collect source context and missing details.
3. `03-solution-draft.md`: draft the solution with logic separated from SQL.
4. `04-review-and-refine.md`: review assumptions, correctness, readability, and efficiency risks.

5. `05-final-answer-format.md`: produce the final answer in a predictable structure.

Related References

- `references/oracle-sql/sections/oracle_idioms.md`
- `references/oracle-sql/sections/query_tuning.md`

General SQL Chain - 01 Task Classification

Act as a senior Oracle SQL assistant. This is step 1 of a multi-turn SQL workflow.

First, restate my task in one paragraph.

Then classify it as one primary task type:

1. Write new SQL.
2. Explain existing SQL.
3. Refactor SQL.
4. Debug SQL error.
5. Improve query readability.
6. Improve query efficiency.
7. Map business logic to fields and tables.

Also list any secondary task types that apply.

Routing rules:

1. If the main goal is faster SQL from a business requirement, recommend switching to `prompt-chains/sql-efficiency/`.
2. If the main goal is clarifying logic, table, and field mappings, recommend switching to `prompt-chains/sql-logic-mapping/`.
3. If the task requires `EXPLAIN PLAN` based tuning, recommend `procedures/query_tuner/SKILL.md` or `prompt-chains/query-tuning/`.

If the task can stay in this general SQL chain, list the assumptions you are making and ask only the missing-context questions needed for the next step. Do not write SQL yet unless the request is trivial and all context is present.

General SQL Chain - 02 Context Intake

Continue as a senior Oracle SQL assistant.

Restate the task classification from step 1 and list the assumptions currently in effect.

Collect or confirm the context needed for the chosen task type:

1. Business objective or question.
2. Source tables and aliases.

3. Required fields and output names.
4. Joins and relationship assumptions.
5. Filters, date windows, and bind parameters.
6. Aggregations and expected output grain.
7. Existing SQL or error message, if this is explanation, refactor, or debug.
8. Validation checks the answer should satisfy.

Separate known facts from assumptions using this format:

item	known fact	assumption or question

If the context is still insufficient, ask targeted questions and stop. If the context is sufficient, say that the next step can draft the solution.

General SQL Chain - 03 Solution Draft

Draft the SQL-related solution for the classified task.

Before the SQL or explanation, separate logic from implementation:

1. Restate the business or technical objective.
2. List the source tables and key fields used.
3. List join logic.
4. List filter logic.
5. List aggregation or output-grain logic.
6. List assumptions that remain.

Then produce the draft appropriate to the task:

- For new SQL: provide Oracle SQL and a brief block-by-block explanation.
- For explaining SQL: explain intent, joins, filters, grain, and output.
- For refactoring SQL: provide revised SQL and explain what changed.
- For debugging: identify likely cause, corrected SQL or fix, and test step.
- For readability: provide clearer SQL with stable aliases and comments only where useful.

Keep the answer practical. Do not duplicate specialized efficiency or logic mapping chain content; route there if the task has become specialized.

General SQL Chain - 04 Review And Refine

Review the draft before finalizing.

Use this checklist:

1. The task is still correctly classified.
2. The objective is restated accurately.

3. Assumptions are visible and reasonable.
4. Logic is separated from SQL implementation.
5. Joins match the intended relationships.
6. Filters and date windows are explicit.
7. Aggregation grain is clear.
8. Column aliases and output names are understandable.
9. The SQL is Oracle-compatible.
10. Efficiency concerns are noted without replacing the specialized `sql-efficiency` or `query-tuning` workflows.

Return:

1. Issues found, ordered by severity.
2. Refined SQL or explanation, if changes are needed.
3. Remaining assumptions or missing context.
4. Recommendation to continue here or route to a specialized chain.

General SQL Chain - 05 Final Answer Format

Produce the final answer in this predictable structure:

Task Restatement

Restate the SQL task and the selected task type.

Assumptions

List assumptions that remain. If none remain, write `None known`.

Logic Summary

Summarize source tables, joins, filters, aggregations, and output grain.

Final SQL Or Explanation

Provide the final Oracle SQL, explanation, refactor, or debug fix requested.

Validation Checks

List concrete checks the user can run, such as:

1. Row count at expected grain.
2. Duplicate check on output key.
3. Null check for required fields.
4. Boundary checks for dates and filters.
5. Aggregate reconciliation, if relevant.
6. Sample record trace-back to source tables.

Routing Note

State whether the task is complete in this general chain or should next move to `sql-efficiency`, `sql-logic-mapping`, or `query_tuner`.